
canvasGUI 3

Inside canvasGUI 3

Introduction

This document supplements the information provided in the source code documentation and the code examples to provide an insight into the inner workings of canvasGUI 3.

Hopefully anyone using canvasGUI will find this guide interesting but for any programmer wanting to add to or extend canvasGUI's functionality it will be invaluable.

Brief history

In 2022 I decided to learn JavaScript and create a p5js library that had a similar look and feel to G4P, a GUI library I wrote for the Processing IDE. This new library would display many of the traditional GUI controls on the actual HTML canvas element giving it a Java-applet type interface. It was first released in September 2022.

Versions 1 and 2 of canvasGUI both depended heavily on p5js making it less efficient than using native JavaScript directly. Version 3 solved this problem by removing all dependencies and using 100% native JavaScript. Although no longer dependant on p5js both p5js sketches and JavaScript applications use identical code to use canvasGUI.

The GUI controller

At the heart of any application using canvasGUI is the *gui-controller-object*. The controller is an instance of the GUI class (File: canvas_gui.ts) and is created with the statement,

```
gui = createGUI(id, display);
```

where the parameters are

- | | |
|---------|--|
| id | a unique string identifier for this GUI. In canvasGUI every GUI control must have a unique identifier to avoid having a plethora of global variable identifiers. |
| display | identifies the canvas element to be used and must be one of the following <ol style="list-style-type: none"> 1) an existing HTML canvas element. 2) the id of an existing HTML canvas element. 3) if using p5js it is the value returned by the createCanvas() statement when executed inside the setup() function. |

Among other things the controller is responsible for

- 1) Creating the controls e.g., labels, buttons, sliders etc.
- 2) Setting the global attributes to be used when creating the controls e.g. colour scheme, font, text size etc.
- 3) Rendering the controls onto the display canvas.
- 4) Processing all mouse, touch and key events associated with the display canvas.

This guide will discuss items (3) and (4) in some detail.

Note: in the following pages the variable identifier `gui` will be used to reference the controller object.

Rendering the GUI

In canvasGUI versions 1 & 2 each control was responsible for drawing itself directly onto the display canvas. It meant that each control required two drawing methods depending on whether the rendering context was '2d' or 'webgl2'. Version 3 introduced the **UI buffer**, an offscreen canvas with a '2d' rendering context so now all controls have a single method that draws onto this buffer.

Executing the method `gui.draw()` causes the gui to be rendered onto the display canvas and has several steps, they are

- 1) If the display canvas has changed size, then resize the UI buffer^{#1} to match.
- 2) Clear the UI buffer^{#1}.
- 3) Cause all visible controls to draw themselves to the UI buffer.
- 4) Render the UI buffer onto the display canvas (the method used depends on the display canvas rendering context).
- 5) If the display canvas has changed size, then update the size and position of all side panes.

^{#1} There is a second buffer the pick buffer (PK buffer) which will be resized and cleared alongside the UI buffer in steps 1 and 2. The PK buffer is used during event handling methods and is discussed later.

Event Handling

Since we are only interested in events associated with the display canvas the following methods are used to add event listeners to the display canvas.

- `_addMouseEventHandlers`^{#2}
- `_addTouchEventHandlers`^{#2}
- `_addFocusHandlers`^{#2}
- `_addKeyEventHandlers`^{#3}

^{#2} mouse, touch and focus event listeners are added when the gui controller is created.

^{#3} key event listeners are only added if a textfield control is created.

This table summarises the event types detected by canvasGUI.

Event types to listen for.	Event handler method. 'e' is the JavaScript event object	Event object ('e') class
'mousedown' 'mouseup' 'mousemove' 'wheel' 'mouseout' 'mouseenter'	<code>_processMouseEvent(e)</code>	MouseEvent
'touchstart' 'touchend' 'touchmove' 'touchcancel'	<code>_processTouchEvent(e)</code>	TouchEvent
'focusout' 'focusin'	<code>_processFocusEvent(e)</code>	FocusEvent
'keydown' 'keyup'	<code>_processKeyEvent(e)</code>	KeyEvent

All these methods are members of the GUI class (File: `canvas_gui.ts`)

Mouse and touch events

The event handlers for mouse and touch events basically perform the same two actions

- 1) Calculate the XY coordinates of the event relative to the top-left corner of the display canvas.
- 2) Call the `_processEvent` method passing the event object and the XY coordinates for further processing.

The `_processEvent` method determines which control(s), based on the XY coordinates and the current state of the gui controller, should handle the mouse / touch interaction.

To do this the gui controller has to be able to identify the control^{#4} under the XY position. Although there are several possible algorithms that could do this canvasGUI uses a colour picking algorithm which is discussed in more detail later.

^{#4} we might only be interested in part(s) of the control. For instance, the ranger has two thumbs and canvasGUI needs to know whether the XY coordinates is over one of them and if so which one.

Focus events

If the user interacts with something outside the display canvas, then our application loses focus. This is detected with the 'focusout' event which deactivates the current control leaving the controller in a *tidy slate* ready for when the application regains focus.

Key events

The textfield control is the only one that responds to key events but only when activated by clicking or touching it. All mouse and touch events are ignored while a textfield is active.

The Pick Buffer

This buffer is an integral part of the colour picking algorithm which is used when processing mouse and touch events.

When creating a control that responds to mouse and touch events canvasGUI generates a unique *pick colour* (32 bit ARGB colour value) for that control, because of this I call it a *pickable control*.

During the gui rendering process the control is drawn to the pick (PK) buffer using the controls pick colour. If we take into account the PK buffer then the steps involved in the `gui.draw` method can be restated as -

- 1) If the display canvas has changed size, then resize the UI and PK buffers to match.
- 2) Clear the UI and PK buffers.
- 3) Cause all visible controls to draw themselves to the UI buffer.
- 4) Cause all visible and enabled pickable controls to draw themselves to the PK buffer.
- 5) Render the UI buffer onto the display canvas (the method used depends on the display canvas rendering context).
- 6) If the display canvas has changed size, then update the size and position of all side panes.

The colour picking algorithm maps the pick colour to its control making it possible to determine which control is under any XY position in the display.

When drawing to the PK buffer it is only necessary to render the areas of the control that interact with the mouse, for example the draggable thumb on a slider. For most controls a single pick colour is satisfactory but there are exceptions, for instance, a ranger control needs 2 pick colours 1 for each thumb to distinguish which thumb is being dragged.

In canvasGUI each control can have 1 to 8 pick colours, which means a maximum of 2^{21} (2,097,152) pickable controls which should be more than enough for anyone ☺.

The Pick Colour

As previous stated the pick colour is a 32 bit ARGB colour value and this diagram shows how the bits are encoded.

Alpha								Red								Green								Blue							
1	1	1	1	1	1	1	1	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*
↑ Most Significant Bit								Least Significant Bit ↑																							

The alpha channels bits must all be set otherwise overlapping controls produce a merged colour making it impossible to identify the correct control. The next 21 bits (light blue) identifies the control and the last 3 bits (light green) identifies a part of the control.

The *control colour key* is obtained by performing a logical AND between the pick colour and 0x00FFFFFF8 (colour mask). The value obtained is represented by the bit pattern -

0	0	0	0	0	0	0	0	0	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

The *part number* is obtained by performing a logical AND between the pick colour and 0x00000007 (part mask). The value obtained is represented by the bit pattern –

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	*	*	*
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

So, the *control colour key* is used to identify the control and the *part number* indicates the part of the control.

Rendering the Controls

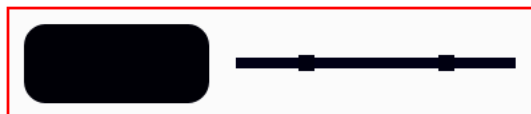
In canvasGUI all controls are responsible for rendering themselves to the UI and PK buffers and once they have all been rendered the gui controller is responsible for displaying the UI buffer on the display canvas. This process is repeated every frame (typically 60fps).

Consider a simple application with 2 controls

- a button with text and icon
- a ranger with tick marks



Each control would require many programming statements to render itself to the UI buffer and while rendering to the PK buffer involves less work both have to be repeated every frame.



Even though the human eye can't distinguish between them, this PK buffer uses 5 pick colours, 1 for the button and 4 for the ranger.

If we consider that

- a control requires many programming statements to create its visual appearance (user interface),
- the application might have dozens if not hundreds of controls,
- the application's purpose is not to display the GUI rather the GUI is there simply to control the applications functionality

then the priority is to maximise the speed and minimise the impact of processing the GUI.

To achieve this goal every canvasGUI control^{#5,6} has its own user interface (UIC) and pick (PKC) buffers and it is these buffers that are rendered onto the main UI and PK buffers. This has the added advantage that all controls have a shared rendering method called `_draw` which performs the following steps

Tags :

UIC the control's user interface buffer

PKC the control's pick buffer

UI the gui's user interface buffer

PK the gui's pick buffer

- 1) If the control has changed size since last displayed then
 - ♦ Resize UIC buffer
 - ♦ Resize PKC buffer
 - ♦ Invalidate UIC and PKC buffers
- 2) If the UIC buffer^{#7} is invalid then
 - ♦ Update UIC and PKC buffers to reflect visual changes
- 3) Save the UI buffer matrix
- 4) Translate the UI origin to the control's xy position
- 5) If this control is visible
 - ♦ Translate and rotate the UI matrix to reflect the controls orientation
 - ♦ Draw the UIC buffer to the UI buffer
 - ♦ If the control has a PKC buffer and is enabled then
 - Save the PK buffer matrix
 - Copy the UI buffer matrix to the PK buffer matrix
 - Draw the PKC buffer to the PK buffer
 - Restore the PK buffer matrix
- 6) For each child control (if any)
 - ♦ If the child control is visible call the child's `_draw` method
- 7) Restore the UI buffer matrix

^{#5} the 'pane' control does not have a UIC or PKC because its visual representation is a simple rectangle it is directly to the UI and PK buffers instead.

^{#6} some controls do not respond to mouse and touch events so do not have a PKC buffer these include label and poster controls. In this situation steps involving the PKC buffer do not apply so can be ignored.

^{#7} the UIC buffer will be invalidated if there is a change in the control's state. For instance, dragging a ranger's thumb would invalidate the buffer.

Lazy UIC and PKC buffer updates



All controls in canvasGUI must have a unique string id and in this application, they are 'btn' and 'rng'. Consider what happens when we execute the following code

```
gui.$('btn')    // get the control's object reference
    .textSize(10)
    .textStyle('bold')
    .iconAlign('right')
    .scheme('green');
```

All four of these statements affect the visual appearance of the button but it would be very inefficient to update the UIC buffer 4 times. To avoid that the control remembers these new

attribute values and only updates the UIC buffer when canvasGUI wants to render the control by calling the button's `_draw` method.

Every control has its own concrete class for instance -

This creates and returns	an instance of the class
<code>gui.button(uid, ...)</code>	<code>CvsButton</code>
<code>gui.label(uid, ...)</code>	<code>CvsLabel</code>
<code>gui.textfield(uid, ...)</code>	<code>CvsTextfield</code>

`(uid, ...)` is the control's unique string identifier followed by parameters specific to the control type.

Each of these classes have two update methods -

`_updateControlVisual()`

This method is responsible for updating the UIC buffer it is only executed if the buffer has been invalidated and the control has been asked to render itself.

`_updatePickBuffer()`

This method is responsible for updating the PKC buffer it is only executed if the control has a pick buffer and the UIC buffer has been invalidated. If necessary, this method is only called during execution of the `_updateControlVisual` method.

Conclusion

Whenever someone creates an application, it is to perform some desired activity and the GUI is simply there to manage the application's actions so should have a minimal impact on overall performance.

Not only is canvasGUI 3 significantly faster than previous versions it has made many other improvements including

- new controls (image button and poster)
- in text-icon controls (label, button, checkbox and option) it is now possible to set the vertical alignment of the text and icon
- many other minor configuration options were added or improved
- improved extensibility

I say improved extensibility because it is now possible for users to create their own controls and have them work seamlessly with canvasGUI.

Finally, I hope you enjoyed reading this guide into the inner workings of canvasGUI 3 and that you found it informative, interesting and inspiring.